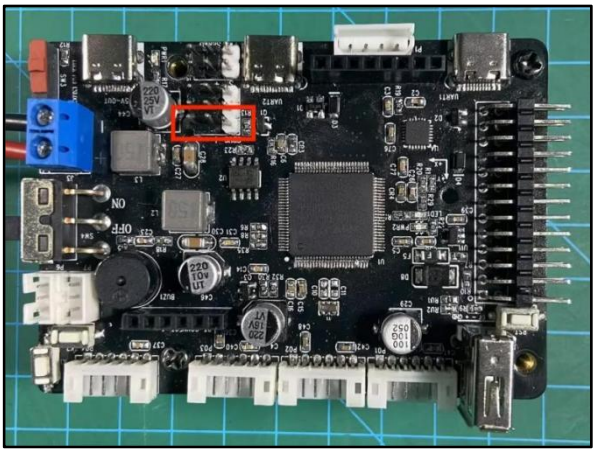
RRC Flight Control SBUS Reception Principle and Source Code Analysis

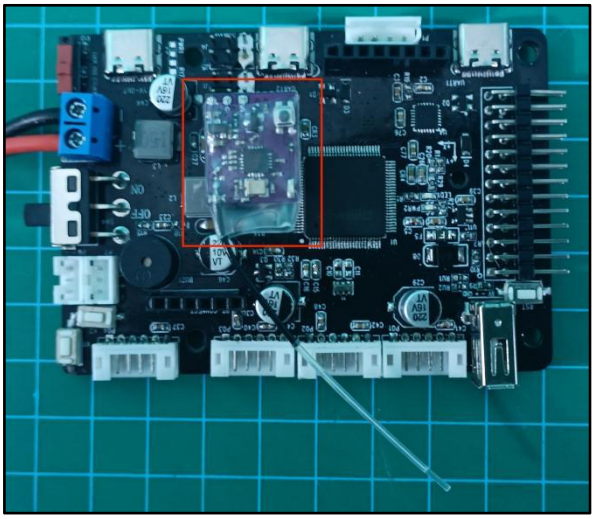
1. Program Logic

This chapter mainly focuses on receiving instructions from the remote control for aircraft models, controlling the movement of the vehicle, and the rotation of the pan tilt.

2. Hardware Connection

The hardware connection diagram of the flight controller receiver is as follows:





3. Program Download

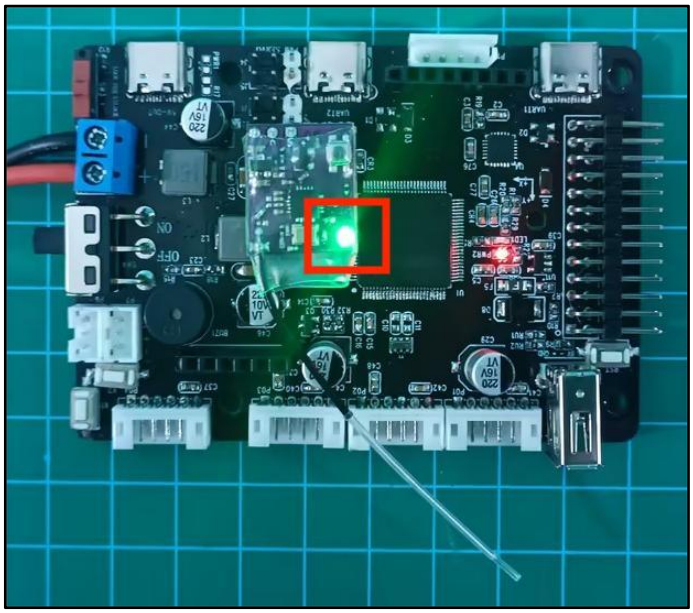
The example routine for this section is '**RRC Flight Control SBUS Reception Routines/（Corresponding Car Model）/RosRobotCOntrollerM4_SBUS_AirplaneSystem**'. You need to burn the program according to the corresponding chassis type.

4. Program Outcome

After the development board is powered on, the flight controller receiver will slowly flash once per second when no remote control is connected;

Pressing the small button on the flight controller receiver will activate the pairing mode, indicated by rapid flashing four times per second;

Next, turn on the switch of the remote control. When the indicator light on the remote control remains solid, the connection is successful.





①: Power switch. Short press to turn on, long press to turn off;

②: Button 1, with three positions. Default to the middle position. Press once downward to trigger a single beep from the buzzer; press upward to activate intermittent beeping of the buzzer, with no action in the middle position;

③: Button 2, with two positions. Press upward to enable left and right translation control of the vehicle using the left joystick (this function applies only to Mecanum wheel vehicles, not to other types); press downward to control left and right turns or spins of the vehicle using the left joystick;

④: Button 3, with two positions. Press upward to lock the gimbal in the center position, preventing rotation control with the right joystick; press downward to enable gimbal selection control with the right joystick;

⑤: Button 4, with three positions, controlling the vehicle speed. Default

position is the middle gear. Press upward once and release to increase the vehicle speed by one gear; press downward once and release to decrease the vehicle speed by one gear;

⑥: Left joystick: Controls the vehicle to move straight forward and backward, turn left and right, or spin. Also controls forward and left/right turning, as well as backward left/right turning. Left/right movement mode is controlled by button 2;

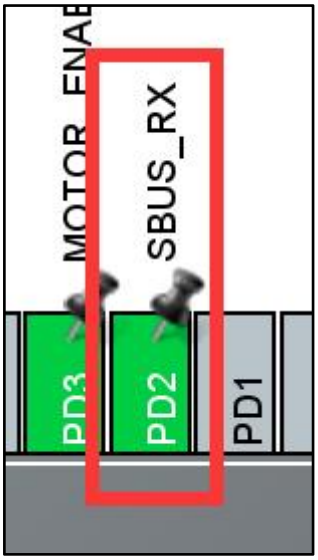
⑦: Right joystick: Controls gimbal rotation. Left/right movement controls gimbal rotation left and right, while forward/backward movement controls gimbal pitch and rotation.

⑧⑨: Adjust the joystick centering parameters. When adjusted, it will emit a long beep. (!!! Users should not adjust it casually!!!).

⑩: Configure the remote control mode. Start from the left and move to the middle position for the first dial, then downward for the following dials. (!!! Incorrect configuration may cause confusion in remote control operation!!!).

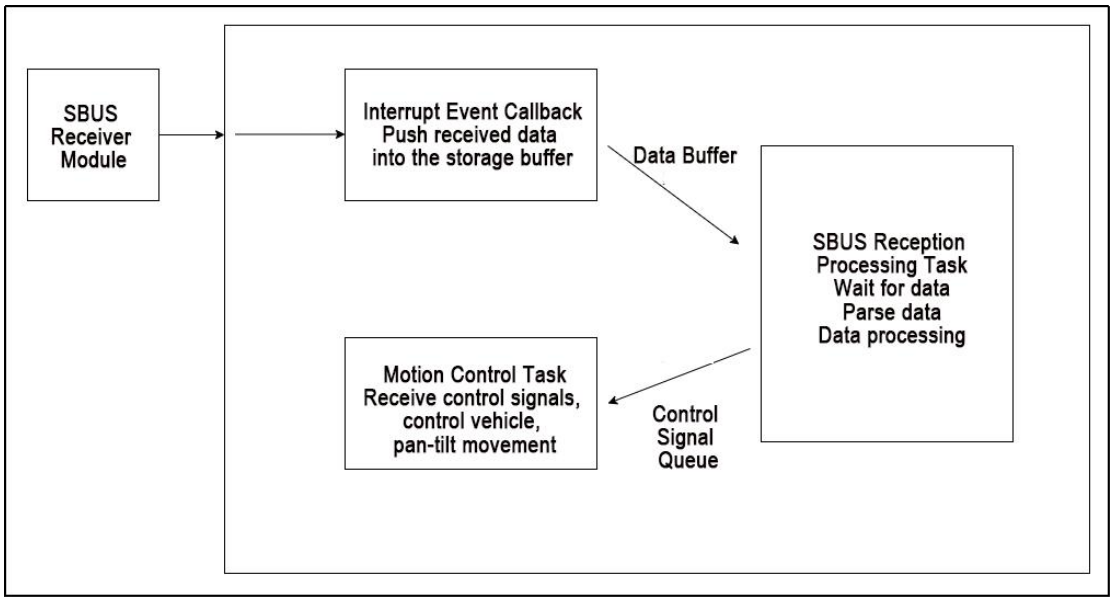
5. PIN Info Configuration

The flight controller receiver communicates with the development board via the SBUS protocol, similar to the UART protocol, and only requires data reception. By using STM32CubeMX software to view pin information, according to the diagram below, the corresponding chip pin is PD2.



6. Source Code Analysis

The program source code for this session is located in '**RRC Flight Control SBUS Reception Principle and Source Code Analysis / RosRobotControllerM4_SBUS_AirplaneSystem**'. The program framework is designed according to the functionality as shown in the diagram below. The analysis mainly focuses on the 'Hiwonder/Misc/sbus.c' file and the 'Hiwonder/Porting/sbus_porting.c' file.



The content is divided into three parts: Interrupt Event Callback Handling, SBUS Reception Processing Task, and Motion Control Task.

6.1 Interrupt Event Callback Handling

The SBUS reception utilizes the hardware UART5 port. In the main.c file, UART5 is enabled.

```
__HAL_RCC_UART5_CLK_ENABLE();
```

The sbus_dma_receive_event_callback() function in the sbus_porting.c file serves as the interrupt event callback function for UART5. It receives data from UART5 and pushes it into the custom data storage buffer subs1_rx_dma_buffers[]. The code is as shown in the diagram below:

```
75  static void sbus_dma_receive_event_callback(UART_HandleTypeDef *huart, uint16_t length)
76  {
77      uint32_t cur_index = sbus1_rx_dma_buffer_index; /* 取得当前DMA缓存的索引号 */
78      sbus1_rx_dma_buffer_index ^= 1;
79      if(length < SBUS_RX_DMA_BUFFER_SIZE) {
80          //终止接收
81          HAL_UART_AbortReceive(&huart5);
82      }
83      //以DMA方式接收数据
84      HAL_UARTEx_ReceiveToIdle_DMA(   &huart5,
85                                      (uint8_t*)sbus1_rx_dma_buffers[sbus1_rx_dma_buffer_index],
86                                      SBUS_RX_DMA_BUFFER_SIZE);
87      lwrb_write(sbus1_rx_fifo, sbus1_rx_dma_buffers[cur_index], length); /* 将接收到的数据写入fifo ring */
88      if(lwrb_get_full(sbus1_rx_fifo) >= 25) { /* SBUS 每帧数据大小为25个字节，  每一次超过25个字节就进行一次解析 */
89          osSemaphoreRelease(sbus_data_ready_01_Handle); /* 置位 SBUS 数据就绪信号量 */
90      }
91  }
```

6.2 SBUS Reception Processing Task

1. SBUS Reception Processing Task

The sbus_rx_task_entry() function in the sbus_porting.c file is the entry point for the SBUS reception task. The code logic is as follows:

1）Initialize SBUS

2）Terminate the UART, set up and register the callback event for data reception

3）Loop:

①Wait for data reception. If no data is received, go back to step 2) to reset; if data is received, continue;

②Attempt to parse the data. If parsing is successful, process the data; if parsing fails, move to the next received byte and start parsing again;

The specific function is as below:

```
void sbus_rx_task_entry(void *argument)
{
    //SBUS接收缓冲区
    static uint8_t buf_temp[32];

    //初始化SBUS
    sbus_init();

    /* 开始接收 */
//跳转标签
start_sbus_receive:
    //中止串口接收数据，防止数据
    HAL_UART_AbortReceive(&huart5);
    HAL_UART_RegisterRxEventCallback(&huart5, sbus_dma_receive_event_callback); /* 注册接收事件回调 */
    /* 使用 ReceiveToIdle_DMA 进行接收，  该函数会在DMA缓存满时中断或在接收空闲时中断并触发接收事件回调 */
    HAL_UARTEx_ReceiveToIdle_DMA(   &huart5,
                                    (uint8_t*)subsl_rx_dma_buffers[sbusl_rx_dma_buffer_index],
                                    SBUS_RX_DMA_BUFFER_SIZE);
    /* 解析循环 */
    for(;;) {
        if(osSemaphoreAcquire(sbus_data_ready_01_Handle, 1000) != osOK) { /* 等待数据就绪 */
            goto start_sbus_receive;  //跳转至 start_sbus_receive 标签
        }
        for(;;) {
            memset(buf_temp, 0, 25); //清空缓冲区
            if(lwrb_peek(sbusl_rx_fifo, 0, buf_temp, 25) == 25) {  /* 从缓存中读25个字节 */
                if(sbus_decode_frame(buf_temp, sbusl_status) == 0) { /* 尝试解析直到字节不够 */
                    lwrb_skip(sbusl_rx_fifo, 25);  //丢弃已读取数据
                    if(sbusl_status->signal_loss == false)  //若已连接
                    {
//                      sbus_print_status(sbusl_status);    //打印遥控器的各个数据
                        deal_SbusData(sbusl_status);    //数据处理函数
                    }
                } else {  //若解析不了，则跳到下一个字节，继续解析
                    lwrb_skip(sbusl_rx_fifo, 1);
                }
            } else {
                break;
            }
        }
    }
}
```

2. Flight Controller Data Processing Function

After the SBUS reception task parses the data, it passes the values to the flight controller data processing function deal_SbusData(). This function converts the changing values into corresponding motion control command values (these command values are the same as those for gamepad control and Bluetooth control) and sends them to the motion control function through a queue.

（1）Process the Joystick Value

The midpoint value of the joystick is 992, with a range of [192, 1792]. The joystick has two data points: one for horizontal and one for vertical movement. When the vertical joystick value is greater than 1400 or less than 600, it is considered to be tilted left or right, requiring the vehicle to perform the corresponding movement. The corresponding motion command is then sent to the moving_ctrl_queueHandle queue using the osMessageQueuePut() function. Upon receiving the command, the motion control task will control the vehicle accordingly.

```
/* 航模数据处理函数 */
static void deal_SbusData( SBusStatusObjectTypeDef *status )
{
    //外部声明运动控制队列句柄
    extern osMessageQueueId_t moving_ctrl_queueHandle;

    {//左摇杆
        static char direction_msg_l = 'I';
        static char last_direction_msg_l = 'I';
        //对左摇杆的位置做映射
        direction_msg_l = A_T_C(status->channels[3], status->channels[2]);
        if(direction_msg_l != 'I') { //若不是停止信号
            //判断 拨钮二 所选的模式  平移或自旋，选择信号
            if( direction_msg_l == 'C')
            {
                //摇杆回中的值为992，范围为[992-800 ，  992+800]，
                //若拨钮有两档，则只有192和1792两个值，三档则为192、992、1792。
                direction_msg_l = status->channels[5] > 992 ? direction_msg_l : 'l';
            }
            if( direction_msg_l == 'G')
            {
                direction_msg_l = status->channels[5] > 992 ? direction_msg_l : 'p';
            }
            osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg_l, 0, 0); //向队列发送控制信号
        } else {
            if(last_direction_msg_l != 'I') { /* 只在松开时发送一次松开信号 */
                osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg_l, 0, 0);
            }
        }
        //记录该值
        last_direction_msg_l = direction_msg_l;
    }
```

There are two types of toggles. One is a two-position toggle: toggling up receives a value of 192, toggling down receives a value of 1792. The other is a three-position toggle: toggling up receives a value of 192, the middle position is 992, and toggling down receives a value of 1792.

The right joystick controls the pan-tilt movement, and toggle switch three provides a mode selection for pan-tilt centering lock. If toggled up, it activates the centering lock mode, skipping the processing of the right joystick values. If toggled down, it activates the joystick control mode, and the right joystick

values are processed. The corresponding pan-tilt control command is then sent to the moving_ctrl_queueHandle queue using the osMessageQueuePut() function.

```
    static char direction_msg_r = 'I';
    static char last_direction_msg_r = 'I';
    if( status->channels[6] < 992 ) //锁定回中  拨钮三控制云台回中锁定、解锁
    {
        if( 'f' != last_direction_msg_r )
        {
            //发送一次回中指令，不让摇杆控制云台旋转，则达到锁定效果
            direction_msg_r = 'f';
            last_direction_msg_r = direction_msg_r;
            osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg_r, 0, 0);
        }
    }else{  //若云台不锁定
        {//右摇杆
            static uint8_t send_count = 0;
            //进行摇杆的位置信号转换
            direction_msg_r = A_T_C(status->channels[0], status->channels[1]);
            if(direction_msg_r != 'I') {
                send_count++;
                //减缓发送信息的速度 尽量控制在每秒发送20次信号以内的速度，能达到好的舵机控制效果
                if( 4 == send_count )
                {
                    send_count = 0;
                    last_direction_msg_r = direction_msg_r;
                    //将信号映射到 J 至 R
                    direction_msg_r = direction_msg_r - 'A' + 'J';
                    //发送控制信号
                    osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg_r, 0, 0);
                }
            } else {
                if(last_direction_msg_r != 'I') { /* 只在松开时发送一次松开信号 */
                    send_count++;
                    //这里是由于信号发送的频率太快，计数4次再发送
                    if( 4 == send_count )
                    {
                        send_count = 0;
                        last_direction_msg_r = direction_msg_r;
                        direction_msg_r = direction_msg_r - 'A' + 'J';
                        osMessageQueuePut(moving_ctrl_queueHandle, &direction_msg_r, 0, 0);
                    }
                }
            }
        }
    }
```

（2）Processing Toggle Values

When toggle one is flipped up (value of 192), the buzzer sounds once every 100 counts to create an intermittent beeping effect. When toggle one is flipped down, it beeps once and stores the current value. If the stored value matches the current value, it indicates that the beep has already sounded once, so the beep action command should be skipped.

```
227     static char msg_l = 'R';
228
229     static int16_t last_data_l = 992;
230     if( status->channels[4] > 1400 ) //拨钮一 当前为往下打
231     {
232         if(last_data_l < 1400 ) //上一次不是往下打，则蜂鸣器响一声
233         {
234             msg_l = 'S';    //
235             osMessageQueuePut(moving_ctrl_queueHandle, &msg_l, 0, 0);
236         }
237         last_data_l = status->channels[4]; //记忆这次拨钮值
238     }else if( status->channels[4] < 600 ) //拨钮一 当前为往上打
239     {   //则蜂鸣器间歇性鸣叫
240         static uint8_t send_count = 0;
241         send_count++;
242         if( 100 == send_count )
243         {
244             send_count = 0;
245             msg_l = 'S';    //
246             osMessageQueuePut(moving_ctrl_queueHandle, &msg_l, 0, 0);
247         }
248         last_data_l = status->channels[4]; //记忆这次拨钮值
249     }else{ //若为中间，则无动作
250         last_data_l = status->channels[4]; //记忆这次拨钮值
251     }
```

When toggle four is flipped up and then returned to the middle, it should increase the speed by one level and send an acceleration command. When toggle four is flipped down and then returned to the middle, it should decrease the speed by one level and send a deceleration command. To eliminate mechanical jitter that causes data errors when the toggle returns to the middle position, a 20-count check is performed to ensure stable acceleration and deceleration functionality.

```
253     static int16_t last_data_4 = 992;
254     if( status->channels[7] > 1600 ) //拨钮四 当前为往下打
255     {
256         if(last_data_4 < 1600 ) //上一次不是往下打，则发送一次减速信号
257         {
258             msg_l = 'n';    //减速
259             osMessageQueuePut(moving_ctrl_queueHandle, &msg_l, 0, 0);
260         }
261         last_data_4 = status->channels[7]; //记忆这次拨钮值
262     }else if( status->channels[7] < 300 ) //拨钮四 当前为往上打
263     {
264         if(last_data_4 > 300 )  //上一次不是往上打，则发送一次加速信号
265         {
266             msg_l = 'j';    //加速
267             osMessageQueuePut(moving_ctrl_queueHandle, &msg_l, 0, 0);
268         }
269         last_data_4 = status->channels[7]; //记忆这次拨钮值
270     }else{ //若为中间，则无动作
271         static uint8_t count = 0;
272         count++;
273         if( 20 < count )  //防抖
274         {
275             count = 0;
276             last_data_4 = status->channels[7]; //记忆这次拨钮值
277         }
278     }
279
280
281 }
```

3. Motion Control Task

The app_task_entry() function in the app.c file is the entry point for the motion control task.

Its content is the same as that of the gamepad remote control. Please refer to

'Lesson 12 RRC PS2 Controller USB Receiver Signal Processing Explanation and Source Code Analysis"